

ActInf GuestStream 085.1 ~ David Bloomin: "[Deep] Learning Active Inference

GuestStream | David Bloomin | 1:23:56

All right, hello and welcome. It's August 5th, 2024. This is Active Inference Guest

Stream 85.1, Deep Learning Active Inference with Love with David Blumen. So thank you,

David, for the presentation and we'll look forward to the discussion.

How we can use Active Inference to make an intelligent agent and some of the advantages and disadvantages and some issues with Active Inference and how I think we can overcome them.

And you'll see why with Love in a little bit. But my background is I'm a software

engineer and an AI researcher. I've worked for a long time at the intersection of scalable systems and machine learning. And I've been working on this project for about three years

now. And I'm really excited to talk about it. So I think the holy grail of AI research right

now is how do we make an AI agent that's both generally intelligent and doesn't destroy humanity.

And pretty much I think most people are working on some facet of these two. And I'm really excited

about the generally intelligent part and also not destroying humanity. So here's how I think

we can do it. But first I want to talk about just like get our definitions. So what's an agent? And

there's lots of people arguing about definitions. I'm going to use a really simple one, which is an agent

is something that given a set of past observations produces the next action. So basically given what

you're seeing now and what you've seen and experienced in the past, what should you do next? So a very

simple formulation of an agent as a function. And by generally intelligent, I don't have a rigorous

definition of general intelligence. I don't actually think there is one. But as an observer observing this

agent, we should be able to say, hey, it's acting smart in a wide variety of environments that it

hasn't seen before. And what that means is that it's able to figure out the environment dynamics,

it's able to make mistakes and learn from them. It doesn't get stuck in the local optima, like knows

when to explore, but then when it figures out regularities about the environment, it knows how

to use them. It does this efficiently. It's not just like trying the same dumb thing over and over again.

It makes plans, it can communicate and track with other complex intelligent entities in the environment.

So it's a bunch of different behaviors. And I don't think there's a really clear way to quantify or like,

talk about exactly what they look like, because different spaces in different environments are

going to have a different idea of what an intelligent agent is doing there. So for now, just like think

of it as this grab bag of capabilities. So where does active inference come in? Well, active inference

in the environment is a way of thinking about an agent interacting with its environment as essentially a,

it's a formulation where any agent can be seen as if it was acting in a self evidencing way. And what

that means is that you can treat any object that maintains homeostasis over time, as if the inside of

that object was minimizing free energy, meaning that it had some, some, you can think of it as having some

generative model of its environment around it from an agent's perspective, its boundary, and it is trying

to update its prior beliefs or its posterior beliefs over the boundary in order to minimize surprise. And so

the way it does that is when it receives evidence from the environment, it adjusts its set of beliefs to

concord with what it thinks it's gonna, what the environment is and what it's doing and where it is in that environment. But it's also actively navigating the environment in order to both get, gather more knowledge and improve its set of beliefs, as well as to move into parts of the environment that it thinks it's going to be occupying. And so this is a really beautiful way to capture what any agent is doing at any scale. So this formulation allows us to think about a biological cell or a robot or a person or an organization, a corporation, a country, anything that we want to draw a boundary around, we can basically say, hey, this thing is kind of acting as if it was doing Bayesian inference over its input data and kind of beliefs over a set of states that it's going to be occupying and it's trying to act in accordance with that. So the problem with that formulation, so the nice thing about that formulation is it's very general, it's very beautiful, it is able to generalize what we see at different scales. One thing that it doesn't let us do is actually tell us how do we build such an agent. And there are two reasons that I think that this theory is not useful for actually building an agent. One is that it treats every agent as trying to approximate Bayesian inference. So as the agent observes evidence, it's trying to infer what that means for its generative model and trying to update its posterior. The problem with Bayesian inference is that it is computationally intractable. And the active inference recognizes this and it basically says the agent is trying to approximately do Bayesian inference by sticking a variational inference bound on it. But in reality, what approximately means is different in every situation. So all agents that we see, all systems that we see, they're doing some kind of computation, and that computation can be viewed as approximate Bayesian inference. But in reality, it's a set of heuristics and hacks that have been discovered by evolution, by natural selection, by software engineers, whatever it is, that can be thought of as approximately doing this. But really, they're running some algorithm, it's just some set of heuristics. And active inference doesn't really tell us how to discover that it just says, hey, anything it's doing can be viewed as approximately doing this intractable problem. The other problem around trying to productionize active inference is that it says, well, the agent is going to be occupying a set of states that its prior beliefs said it's going to be occupying. So, you know, why does an animal eat? Well, because evolution has filtered animals down to only the animals that believe that they're going to be fed. And so then the animal is going to be a self evidencing way go seek out food. And again, this is a really beautiful way to capture both the action and inference in one formulation. But it doesn't tell you what are the priors over states for a given environment. So if you want to build a robot or an agent that has to interact with some environment, what what are its prior beliefs? What states should it be occupying? Does it believe that it's going to get energy? Or does it believe it's going to go repair? You know, the car that you want it to repair? How does it balance between those two? We don't know. Active inference doesn't really tell us it just says some process will give you a set of priors. And so with active inference, typically the approach is, well, as a software engineer, you build a generative model, you basically code up what you think the environment requires and the agent in the environment requires and you give it some priors like you will be fed and you will repair this car. And here's some parameters about how the environment works that are simple enough that you can then do a variational inference over it and get

something done. And then you have a more generic free energy minimized minimizing algorithm that that given that generative model actually controls the robot. The approach I want I'm taking is, well, instead of someone trying to design this general model, can we learn it? And I think that's really the only way forward because any environment that we care about is going to be so complex that I just don't believe humans are going to be able to specify a generative model for it that's at all interesting.

So why do I think we can learn it? Well, one evolution has. And what does learning it mean? Learning means that we want to use some learning system and neural nets are a really good one. And we want to figure out what are the sets that we basically want to have some approximate active inference agents, some approximate generative model that is able to do all the things we want, which is that grab bag of generally intelligent behaviors. And the reason we can do that is that neural networks are universal function approximators. So rather than building an algorithm, we took a neural network, we fed it, we fed it pairs of text and images and outcomes, DALLI that given text is just able to generate images. And also generated all the slides for this presentation.

So the way to think about neural networks is rather than programming an algorithm, you figure out the training data and a loss function, and then you take a big blob of learning, learnable parameters, and you keep nudging that blob until it does what you want it to do. And this technique is really powerful. As we know, we're working language models that everyone is using now. We're also trained this way, you took a data set that

was a set of prompts, and a set and then a completion, basically the next word. And you trained a neural network to

predict the next word and you got chat GPT. So this technique is extremely powerful, it is very slow, it requires a lot of data and a lot of

training examples. But over time, you're able to learn a really complicated function approximator. So the question then is,

okay, well, if we want to train a function approximator for a generally intelligent agent, what's the training data that we need to use?

And basically, I think if we discover the training data, and we throw a large neural network at it with a lot of compute, we should be able to in the same way that we got

chat GPT or Dolly, get a model that's like a foundational general intelligent agent model. So to do that, we need an environment. And what an

environment, so the reason we need an environment is that we're trying to train an agent, not say a language model. So for a language model, you give it a prefix, it gives you the completion, and you can train it on, say, all of the internet text. What we're trying to do is we're trying to learn a function that given a set of fast observations, know what action to do next. And that's a more complicated

problem, because sometimes the action it takes should be done because it's trying to accomplish something, sometimes it's because it's trying to learn something, sometimes it's trying to

explore. And so what we need is a world for the agent to interact with, that it can then learn its behaviors from. And so the properties we need for that environment is one, we want it to provide continuous learning opportunities. So there should always be something that the agent is a new thing for the model to learn. And then when it learns it, when it does something that it's

intelligent, one of these behaviors that we associate with intelligence, we want we need a feedback signal. So we need the environment to essentially reward it. And it needs to be an environment that cannot be mastered. Because if it can ever be mastered, then there's nothing new to learn, and the model stops learning. So the way to set that up is, or one way to set that up is via multi

agent. So if you have an agent. So if you have an environment, even if it's a relatively simple one, like say, a go game, where the rules of the game are really simple, just by having other agents in that environment, gives it a really important property, as the agent gets smarter, so do the competitors. And so the environment essentially adapts to your capabilities. So this is a way where you can automatically get an environment that always gets

something new for you to do. And so as soon as you learn some exploit or some way of being better than your competition, the competition changes, and now you have to learn something new.

And so I think, so my approach is leveraging this characteristic, and essentially training AI is in a multi agent competitive environment. There is a problem though, which is if the environment is entirely competitive, you very quickly end up in this killer be killed dynamic. So you can imagine an agent learn some behavior policy, and you can

see and then the other agent learn some counter strategy and then they kind of explore the area around the strategies and figure out the best way to be competitive and other stuff because if any agent ever tries to do anything that is not that it gets exploited by its adversary. So this setup makes it really hard to avoid local maxima, you kind of find something that works and you're stuck there. And that's not quite true. If you have many agents, you might get

some unstable symmetry breaking and get some unstable symmetry breaking and things get smoothed out a little bit. But I think the solution to that is actually cooperation. And what's cool about cooperation is there are many ways to cooperate the space of effective behaviors when it comes to cooperation just explodes.

And so you end up with a much denser more higher dimensional behavior space where it's harder to ever get stuck at some Nash equilibrium of behaviors.

And this also gives rise to complex group dynamics, like coalition building cooperation, cooperating to compete, figuring out who you're going to cooperate with and who you're going to compete with building trust. And it's just like the set of behaviors that are available in cooperative environments explodes. And the problem with cooperation is it's really hard to learn if all the agents are selfish. It's not impossible. You can have things like reciprocal altruism or even just non-aggression.

You can learn that always picking a fight is not great because even if you win the fight, now you're weaker and someone else can take advantage of you.

But in general, learning cooperation when everyone is not cooperative is also full of these Nash equilibrium of local optima.

And nature solves this using kinship. So when an animal is born into a world, it is not surrounded by other selfish organisms.

It's surrounded by a lot of kin. So for cellular organisms, it's surrounded by more or less identical genetic clones of it.

For animals, it's surrounded by organisms that have half of its genes. And this kinship via genetic evolution gives rise to organisms that are not selfish because from a gene's perspective, having a copy of you in one organism or in another organism doesn't matter.

And so what that does is it gives rise to organisms that basically care about the success of other organisms

almost as much as they care about the success of their own organism.

And there's a whole spectrum in nature from hive insects like ants where from a soldier's ant's perspective, the only way for those genes to make it into the next generation is for the queen to reproduce.

And so you end up with ants that care a lot more about the hive and the queen than they care about their own safety and well-being.

And you get a whole spectrum from fully altruistic hive insects to a few perfectly selfish sociopaths.

But most animals fall somewhere in the middle and being in the middle where you're partially selfish and partially altruistic gives rise to all of these social behaviors that we associate with being an intelligent being.

So animals coordinate, they negotiate, they build trust, they love each other, they show kindness, they show jealousy, they have social emotions like shame, they learn how to exchange knowledge with each other via gossip and reputation building, they do division of labor.

And all of these things happen again at every layer of complexity, animals do it, biological cells do it, organizations do it.

These set of sets of behaviors are present in pretty much all entities that we think are intelligent.

And so my proposal back to this AGI recipe is that if we can have the right training data and enough compute, we can train a large model on an environment that essentially balances cooperation competition.

And so I've been trying to build and design these environments, this environment and the training infrastructure for it.

And that's been essentially my research direction.

So this is the current version of the environment.

And I'll just talk through some of the components.

So you can see there's a few agents here and they're they're parameterized by a neural network.

So all of this was trained from scratch using reinforcement learning.

There is this heart altar in the middle.

And the only source of reward that these agents have is they can walk up to the heart altar and they can use it, which requires energy.

And if they do that, they get a reward.

And what that means is that these agents have to get more energy out of their environment than they're using for interaction with the environment.

And they can extract more energy than they need.

And they can essentially waste it by putting it into this heart altar, then they get a reward.

So this is the competitive dynamic that's set up.

So everything doing everything in this environment requires energy to get energy.

You collect these resources.

So you walk up to a green or yellow crystal and you collect the resource.

You then have to walk up to these converter stations that look like little batteries.

And then you put the right set of resources in there and you get energy out.

You can also attack each other.

And if you get shot, you turn into this egg where you're stationary for a while.

You lose some time and people can steal whatever resources you're carrying.

But you can also turn on a shield, which also costs energy.

But then it deflects the attacks.

So these agents have to from scratch learn how do you navigate the space?

Where are the resources?

How do you manage your energy budget?

Who do you fight with?

Who do you not fight with?

When do you need to keep your shield on?

So a lot of like relatively intelligent behaviors that are learned from scratch.

And all of this is done via this competitive dynamic.

And then on top of that is a whole system for kinship where every agent shares its reward with some other agents based on some reward sharing system.

And so what that means is whatever, say, this red agent gets a reward, the green agent might get 10% of that reward.

And so this environment allows you to train agents in very different environments.

So I'm going to pause this and switch over to this other tab.

So yeah, here.

So you can train in larger maps, you can set up different game rules and different game scenarios.

And you can also set up different kinship environments where you can train fully selfish agents, you can train fully cooperative agents, you can train complex non symmetric social dynamics.

And once you train an agent, you can actually start treating it like an intelligence.

And you can do behavioral studies on it.

So here, we take a trained agent that was trained in this environment, and we just like put it in an empty room.

And we're like, well, what what is this agent doing?

And you can see it's like it's exploring its environment.

It's not getting close to any walls, because it knows that there's not a lot to be seen, and it doesn't want to waste its movement.

You can put it in an environment where, you know, you give it some energy and you give it and you put it in a world where there's nothing except this heart alter.

And you'll see that it like searches the world until it finds the heart alter, and then it puts energy in there.

And so you can start doing, you can essentially study it and do experiments on it.

It's like, what will it do under this circumstances?

And you can start approaching some of this as an observer observing an intelligent entity.

This is all early days.

So this is just a few evals and experiments that I've been setting up.

But this environment allows you to do all of these things.

And I think what's cool about this whole approach, and I'll switch back to this slide, is that the idea is that you end up with an environment that is unlearnable, because anytime some agent gets some competitive advantage, the environment around it changes since all the other agents now arbitrage away that advantage. It's also highly, highly dense in behavior space because there's so many different ways to cooperate and compete with each other that there's always some new trick that you can discover.

And so as this neural network learns to achieve fitness to this environment, and the environment is always changing, it's also learning how to learn.

It's learning this set of adaptive behaviors of exploring, forming theories, testing things out, because that's the behavior that gives it an advantage when the environment is always changing.

And so the hope is to end up with an agent that you can drop into a new environment that it hasn't seen before.

And the thing it'll do is first figure out the dynamics of the environment, start exploring it, figure out how different things work, and then what it needs to do, and then start doing it.

The other cool thing about this setup is because these agents are trained in a cooperative setting, they have to cooperate with, they have to learn how to cooperate with other agents.

And that means they have to learn how to communicate.

So there's a signaling and a language component.

So if you are surrounded by kin and you have to compete with another group, you have to learn what information is necessary to communicate to your allies in order to effectively organize.

So these agents are developing a language that we can then be able to reverse engineer because we have a fully instrumented world and we can do experiments on them.

So we can basically figure out what it is that they're saying and we can then talk to them.

And they're learning a policy that's conditioned on this kinship score.

So they know how to cooperate with close kin, which means they are capable of cooperating.

They're learning to be capable of cooperation.

They're essentially learning to care for other agents.

And so the hope of this research program is to build a model organism of an active inference agent, essentially an agent that is performing active inference in a complex environment that you can talk to and that cares about other agents in that environment.

And if that works, we should be able to take these same ideas and take it out of this simple 2D grid world environment and essentially do the same thing, but in complex 3D world physics environments, get something that is smart, that cares about other entities and that we can talk to.

So that's basically what I'm trying to do.

I'm calling this project Meta learning.

Meta is the poly word for love and kindness, because I think that love is love.

What I think I'm building here are agents that are capable of love in the same way that genetic evolution, the selfishness of a gene being expressed in a multitude of other organisms gave rise to organisms that love and care for each other, especially their kin.

I think that similar dynamic is what's necessary to create an environment that's complex enough to learn generalized intelligent behavior and also is going to create agents that are alignable, that basically care for

other agents that we can get along with.

So I'll stop talking here and take questions and happy to dive into anything that I've talked about or if you have questions about active inference or how or free energy or deep learning, basically anything that this made you think about.

I'm so happy to take questions.

Awesome.

All right.

Anyone in the live chat, please feel free to ask a question.

Just meanwhile, maybe just how did you come to be applying the neural network approaches in this project?

Just what led you want to explore this project?

And then how do you see active inference playing a role?

Yeah.

I mean, it's funny because I've moved in and out of machine learning my entire career.

And the experience I've had was I would go into AI.

I would spend a bunch of time working on it and then I'd be like, okay, the field is not ready.

Like, we don't know what we're doing.

And I took a long pause and worked at a startup for 10 years.

And after leaving that startup, I came across Carl Fristen's free energy work and had been out of AI for almost a decade, missed the last two revolutions.

And reading it, it just kind of clicked for me.

I was like, okay, like I can see how because this describes intelligence at every scale and just very naturally falls out of what it means for objects to maintain homeostasis in an entropic environment.

And I can see what the math looks like.

The math looks like essentially the math from reinforcement learning.

It's trying to do basing and inference on your evidence to kind of compute a posterior and navigating the environment in order to balance exploration exploitation.

And that's kind of what the reinforcement learning community came up with bottoms up, just like trying to be like, hey, how do we train an entity that learns in its environment?

And when I saw just how close those two approaches were to each other, it really made me feel like, okay, there's no secret sauce.

It's like there's not something that we're missing that's key to intelligence that we just like haven't been able to capture.

Really, it is like this kind of exploration, exploitation, inference combination.

And now it's just like, how do we build it?

And the more I started thinking about this, the more I was like, well, active inference doesn't tell you how to build it.

It tells you that a bunch of these things have to be either filtered for or discovered.

And reinforcement learning kind of starts out with the idea of like, well, we don't know what we're doing.

How do we build it? How we discover it?

And so then I started exploring the whole spectrum between the two.

You know, do you start out with a world model that you learn, but then you have a set of algorithms that use that world model in some predictable way?

Or do you learn the entire thing as a behavioral policy end to end?

And I see these as essentially pragmatic, pragmatic engineering decisions.

But just like having the confidence that this approach is essentially what all intelligent entities are doing really pulled me back into this field.

And once I realized that what an agent is really doing is not some idealized version of active inference, because the idealized version is computationally intractable, that really led me to be like, okay, it's not doing this perfect thing.

It's doing this approximate thing.

So if everything is doing an approximate thing, and we have the technology to learn an approximator, we should just be able to do that.

Just like have a thing that learns the set of hacks and heuristics that any actual agent is going to have to discover.

And we have the existence proof of being able to do this in other problem spaces like language.

Awesome.

Awesome.

That's super informative.

I'll just let you have a sip of water and kind of restate a few of the key pieces because I think these are really key.

So because free energy principle and active inference describe like any slash every scale, it has come across some of the same issues that reinforcement learning has come across on the low road.

That's why active inference is like between the high road and the low road because of how the architectures are.

It's not just a specific architecture.

It's almost like as you're pointing out because of the fact that it's already an approximation because even the analytical part is an approximation.

The analytical part is variational inference.

So already talking about an approximation to a perfect map.

So at that point, it really is pragmatic.

Like you described, whether you approach this in like a well-structured way.

And there are often an active inference.

People like talk about like structure learning and adding and shrinking parameters and hooking up different things, doing combinatorics with base graphs.

And that may be super promising for certain pieces and like keeping it well typed and well defined.

And then the other approach, which is resonating with with how you're saying big problems and other domains have been solved, is like actually an interestingly open ended approach, which is the like you said, reinforcement universal function approximator.

So it's a complimentary approach to try to use a statistical approximator of the variational inference.

So my first question is, how about that reward function?

Can you specify different ways or have you specified different ways?

For example, the reward could be when two people interact and don't have a conflict or different things, because just from a sort of predictability perspective, it's like it's actually a power preference centralization.

So you've taken a really flat, uniform, informative approach to any distribution except for one and loaded all the hopes and dreams on one.

So what do you think that does with the architecture?

Yeah, great question. I want to respond to that.

First, I want to respond to something you just said a little bit earlier in terms of like structure models as the generative model for active inference.

And I think what's cool about the free energy principle is that it just like lets you rephrase anything that a system is doing as if it was it had some generative model and it's minimizing free energy on it.

So any any behavior can be rephrased as a as the energy minimization over some set of some generative model over some set of beliefs.

And so it's like a really nice way to just go between two two ways of formulating a problem.

And I think the people that try to take the more principle, less design a generative model for it, the challenges they run into is, well, whatever model they're designing, whatever graph they're designing has for in order to be computationally tractable with variational minimization is some sparse graph of random variables with some sparsity constraints.

And then they're like, well, what what are the what does this graph look like?

And either they're making it up or they're trying to learn it.

But if you're trying to learn it, you're kind of back to this thing of like, well, we don't know what the generative model is.

Let's try to learn it.

But so I see these approaches as kind of being equivalent is just like how much prior structure do you force onto the system.

And then when you actually use so when you do say variational base, what you have what you're saying is, look, I have this structure and I'm not saying this structure represents reality.

But I think this can represent reality to some error.

And then I have this inference mechanism that I'm applying to this structure.

And together we are representing reality, some approximation of reality with some error.

And what's cool about being able to do doing an end to end thing where you're just like, throw your hands up and be like, I want to learn the whole thing is that the set of the set of inference algorithms that you get to explore are also more powerful.

So for example, you can imagine parts of the model where it's like really important to get some detail right.

Whereas other parts of the model is like, it's not that important.

You know what this random variable says, it's like it doesn't have a lot of effect on the downstream task.

So when you're doing free energy minimization, you are you don't get to choose which part of your model is important, how much compute to spend on different parts of it.

Whereas when you're learning an algorithm end to end, part of the thing you're learning is when to do how much inference and how much your inference matters.

So I think this is another kind of way in which the more principled approaches pull in an assumption that maybe is a reasonable assumption, maybe is not, but is definitely limiting.

Yeah, if I can just reiterate, I think that's very farsighted of a way to understand this, because let's just say we're using an rx infer.jl strongly typed Julia language model.

So if you're using a rx infer.jl, the decisions about when and how to optimize the message passing timing and when it would itself, maybe it's irrelevant to the question or maybe you don't need the most efficient.

But if you need open endedness on how the graph is orchestrated, then it just shows that in realistic, even small settings, the structured method is embedded within an unstructured method.

Now that's either a manual curation, like which might happen for small scale.

But if you're talking large scale generative model, then even that outer orchestrative level is essentially statistical.

So that's, that's a really important thing to remember.

And all these other like kind of pragmatic considerations about the data infrastructure and all these types of things, which are like, again, they're not in the textbook, but to get above even a small size run, they start coming into play.

And now I want to talk about the reward question that you asked.

And I think that's a really great question.

This is actually so right now, and this is early days for this project, but right now the reward is this heart alter and plus kinship.

And the thing that I wanted to do with reward is we need some reinforcement signal from the environment to essentially reinforce the policy.

So the policy needs to learn from something.

It needs some learning signal.

And what I wanted to do was make a system where you don't have to keep messing with the system, you just get to scale things up.

And so you make an environment and then you just scale up the size of the models in the environment.

And as we do that, the environment gets more and more complex in an open ended way because the environment is really the minds of the other learning agents, not the physical game.

And so the thing I wanted out of the reward was something that binds the agents to each other and to the environment.

And so it has to be some kind of competitive dynamic.

Essentially, there has to be something that when one agent is doing better than another agent, essentially there's some zero sum finite reward to go around so that there's pressure on the agents to be smarter.

And by smarter, meaning figure out a way to behave that gives it an advantage over some counterfactual other agent.

So that's the idea behind this reward.

The other thing I wanted from the reward is I wanted it to be really sparse so that you're not constantly tinkering around like, oh, I want to, you know, I want these agents to do something more interesting.

I want them to cooperate more.

Let me add a reward for cooperation or like for this interaction.

I basically I don't want to be in the in the game of environment design and reward tinkering.

I want something that's just and I think it's this is a really beautiful reward.

It's essentially free energy.

It's, you know, from an agent's perspective, their job is to extract energy from the environment and then use it to compete with each other.

And then if you ever find yourself with excess of energy, that's your reward.

This is you get this in evolution where it's like, hey, you can get enough energy to reproduce.

If you can muster enough energy to make a copy of yourself, that is the reward for the genes.

That's how the gene gets into the next generation.

So you can think of this as a reproduction reward.

You can think of this.

Partially what I'm doing is kind of re trying to pull the ideas from biological natural selection into reinforcement learning.

And so this is from a genetic perspective, when you make a copy of yourself, it means you got enough energy out of the environment, you made a copy of yourself.

And so the genes get reinforced.

Those are the genes that then persist.

Similarly here, when you got more energy than the competition and you went to put that into the heart alter.

It means that the policy is reinforced.

And so those are the policies that you're going to see.

Originally, I didn't even have the heart alter.

I just had an action that an agent could take that burns energy and gives that reward.

So it was essentially self rewarding.

And I did the heart alter just because it was like easier to visualize as a human watching it.

It's like, oh, yeah, they walk over there.

They do the thing.

You can think of it as like laying an egg essentially.

And so the other aspect of the reward is the kinship.

So when you get a reward, other agents are getting some of that reward as well.

So your source of reward is not just walking up to the heart alter.

It's also helping other agents do it and learning who those agents are and under what circumstances you should help one agent and not the other.

And at what point should a resource go to some other agent and not to you.

So the reward function is actually really complex from an agent's perspective because it is this social reward rather than just a selfish one.

And then one more thing I'll add, which is like really inspired by active inference that I haven't yet implemented, but is very close on my roadmap is one problem with a sparse reward is it's really hard for

agents to learn because a lot of times you're doing stuff and you're not getting any feedback.

And then sometimes you get a reward, but it's not even something you did.

It's something some other agent did that's your kid.

And so this is a really hard learning problem in reinforcement learning.

And one thing that active inference gave us is this idea of surprise minimization as an auxiliary reward.

So a reward that I plan to add to an agent is that there is going to be some small amount of reward that you get just from being able to predict your observation at the next time step.

And what that allows you to do is and it's going to be a really small reward because you don't want to overwhelm the competitive landscape.

You don't want the agent to just be, you know, all I care about is predicting, you know, hiding in a hole and predicting my environment.

You wanted to engage with the rest of the world.

But what this gives you is a very dense learning signal essentially at every time step, even if you're not getting some reward from the environment, you're able to learn some environment dynamics because there's this reward signal about just like figuring out how the world is working.

And I actually think that at the limit, you don't need this because at the limit being able to learning a policy that is able to get this competitive reward requires that you learn a policy that's also a good world model that is always predicting what's going to happen with more weight on the things that actually matter.

So it's like a pragmatic world model instead of just like, you know, I want to predict all of my observations world model, which is why you can't really give it give the predictive reward too much of a weight because not all the things that you're observing matter.

There's an infinity of things you can predict about your environment and you don't want the agent to be trying to predict them all.

You want to be trying to predict the things that matter.

But I do think that it'll make agents that learn faster, at least initially when they don't know how the environment is behaving.

Yeah. Again, a huge point. Even epistemic value, you at some level have to do statistics to figure out the right balancing between epistemic and pragmatic.

All right. There's a bunch of questions in the chat, so I'm just going to give you these questions in the order.

So feel free just to give a response as you see it. OK.

Board guy one one two wrote in systems that lock the architectural capabilities for temporal or counterfactual depth.

How does the process of minimizing free energy relate to their form of epistemic foraging behavior?

How is collective epistemic foraging different with increasing like agent level sophistication in terms of not being able to plan just like one step planners versus like being able to do more planning?

Yeah, I think this is an interesting question.

And I think maybe I missed the first part of it, but I can definitely respond to the second part.

I don't know if there is much of a difference.

An agent is something that takes its past observations and then produces the next action.

And I don't think there is a lot of difference between an agent that is planning versus an agent that is not

planning.

The question is, how sophisticated is the action going to be?

So and you can think about this, you know, you can have let's say you have a thermostat.

Right. People don't even want to call a thermostat an agent.

Like you have something that's trying to control, you know, maintain temperature in the room and you can have a planning thermostat that says, OK, actually, you know, I'm going to make a plan.

If I increase the heat by this many that here's my differential equation.

You know, here's my model of how that's going to change the condition.

So I'm going to instead or you can have a thermostat that just like learned a really good function that has that essentially it unrolled the planning.

It's just like, you know what, in this case, I'm just going to increase the temperature by this much because that effectively is going to turn into the right homeostasis maintenance.

So the way I like to think about it is you have a system and it has some compute available to it and it has past observations and it's going to produce the next action and it's going to use that compute in some way to figure out what that action is.

And planning is a way that we can kind of characterize what is it doing with that with that compute?

Does it have some world model that we can understand and it's doing some future rollouts on it or we don't know what it's doing.

We can treat it as if it was planning, but really is just like, is it able to leverage the computer?

How much compute does it have?

And is it able to leverage that compute into achieving its goals better or worse?

So I kind of think of an agent that is making decisions of just for the next time interval as not that different than an agent where you can explicitly say, oh, it has a world model and it's doing planning on it.

It's just, it's doing some computation and it results in some confidence in achieving its goals.

Yeah.

Great points.

It projects onto the next action.

However you see it and however it's done.

Okay.

Courtney wrote really cool work reminds me of things like the Sims without humans messing up their lives.

Might the agents eventually have some kind of introspection in their model in a more advanced simulation?

I suspect that I can look at the more and more of a bigger and bigger part as we train bigger and bigger networks.

works. So an agent that's trying to do something needs to essentially have some trajectory.

If an agent is trying to get food, it needs to keep, it's going to do a step towards the food, and then it's going to have to do the next step towards the food. So it already has some kind of internals that give it consistent behaviors. It has some set of beliefs around, or it can be modeled as having some set of beliefs around what its state is, what its goal state is. In order to pursue that goal, it has to have theory of mind of other agents, right? The environment it works in,

like whether or not it's going to get its food depends on what all the other agents around it are going to do. Like, is it going to get shot or not when it's trying to go over there? Does it need to turn on its shield? Does it need to turn off its shield? So a lot of the things that the agent is modeling are the states of other agents, and the states of other agents depend on its state. Maybe if it has a shield on, it's going to get shot, but if it has a shield off, it's not going to get shot. So it has to be, in order to predict what's going to happen to it, it has to know what other agents are going to do, which is a function of what it's doing. So I think that's key to just like the whole thing working at all. So I think introspection is there. The real question is, how do we, as external observers, start characterizing it having introspective abilities? And what experiments can we perform to start quantifying? How much introspection does it have? And I think that's really a challenge, is not what is it doing, but how do we figure out what it's doing? How do we distinguish an agent that has introspection from one that doesn't, or has more introspection or not? And that's like a whole research agenda that I think is like really interesting, and I would love help with. Awesome. All right. John Klippinger wrote, can you have cooperative learning and reward rather than zero sum a la Martin Novak Shapley value? I mean, yeah, I think cooperative learning is what's happening here, because no agent's reward is a zero sum reward. I mean, there is a zero sum reward over the whole system, because there's finiteness of resources. And I think you can't get away from that. All of existence, all of reality is essentially allocating finite energy to infinite potential compute. So there is a zero sumness nature to everything. What's cool about this cooperative learning dynamic is that from every agent's perspective, it's not an individual agent, it is some smeared agent over other agents, what it cares about what it's trying to maximize is some linear combination of other agents, including itself. So all of the learning is essentially cooperative learning. From an agent's perspective, a close skin agent doing something that gets it a reward is the same as it doing it. So we're kind of blurring the line between, you know, is this other agent just your left hand versus your right hand? Or is this your child, brother, parent? Or is this a complete stranger? And I think the learning gets smeared across all of those regimes?

Yeah, there's so many more things. I'll just make a short comment and ask one or two more questions. I see it as you're using modern methods to bring in a kind of mixed or multi game theory, because there might be some Nash theory or clean analytical solution to like some single dimensional game, like just this kind of game or just that kind of game. But when there's like tools, you know, you're going to be able to use opportunity cost, co visibility, it's not as simple as just like, as like, just buying and selling on one uncertainty. So it then also when you consider the niche and niche modification through pheromones, as well as through the collective defense, in like you social insects and naked mole rats, and all of that kind of convergent ecologies, leading to evolutionary transitions in the multi scale system.

And then the further like, and then the further like, canalization of that, for example, the nest mate workers who don't develop ovaries at all. So at that point, calculating these partial reproductive scores is is non functional.

Okay, I'm just gonna read a few of the comments and then um, but first just what, what are your last or I

mean, sorry, what are your next moves? And then I'll just read you these the rest of the comments and you can respond anything but just where does this go slash what where do you where do you hope people can participate in everything?

Yeah, great.

Yeah, great. Thanks for that question. And I'll just throw out like I don't have a hard end time. So I'm happy to go for okay, because you're down.

Yeah, so my general plan is I'm approaching this as an engineering problem. And it's an open source project. And what I want to do is iterate on the environment, iterate on the training infrastructure and iterate on the agent architecture, and iterate on the evaluations. And I'm kind of approaching this like very improvisationally, I'm just like, okay, what's the

what's the current bottleneck? Where do I think I can get progress? And I would love collaborators. So part of where this like part of how I think this project is going to work is based on who I can get to help along with it. But the surface area of the project is huge. It's every every aspect of it is a bunch of open ended research direction,

like, like, it's really exciting, it's like, it's full of low hanging fruit, it's like, for every one of these directions, I have a bunch of ideas that are all relatively easy that I'm pretty confident are gonna lead to really cool results. So there's a bunch of stuff with environment design that is really fun. So

the environment you saw is really complex, really simple is just like a few little game objects, I have a whole library of new game objects that I want to add, that will really change the dynamics. So the new version of the environment that I've been building is entirely editable. So the agents can build and destroy all objects while preserving total energy in the environment.

I have objects around like long range communication. I mean, there's just like so many little game objects that you can add that really alter the nature of the environment. There's also because everything that you do costs energy. You can get so right now when you train every training episode, the energy costs for all the actions are sampled from some distribution. So sometimes movement is really

expensive and you have to learn how to live in that world. Sometimes attack is cheaper than defense, sometimes it's more expensive. But one thing that I'm really excited is to actually give different energy profiles, essentially different bodies to different agents so that the environment then becomes heterogeneous where different agents have different capabilities.

hey, they want to reward share with you and not some other agent, which gives rise to signaling and counter signaling and lying. And I think was also like really instrumental to runaway intelligence and human and primate evolution. So there's, that is to say, there's a lot to do on the environment side with like lots of really low hanging fruit there. Right now, I'm working on environment performance and rewriting the environment and using a much more performant architecture. So currently, I'm training my models that around 40,000 steps per second, which is pretty huge on a single machine for reinforcement learning for people that are not really involved. The new environment, I'm hoping to get more like a million steps per second. So it's just going to mean that we can train bigger models faster and really improve the iteration speed for a lot of these experiments. I have, so that's on the environment side, there's the training infrastructure, there's a few really cool algorithms that I want to implement for distributed training,

so we can train over many machines. Again, just trying to ramp up the training.

Evaluations is another thing that I'm trying to develop is like, how do we, once we have an agent, how can we tell, you know, is it smarter than the last agent we built? Is it smarter than an agent that has more or less world experience? How do we quantify that? So there's a bunch of stuff around evaluations and cognitive tests and behavioral tests for these agents that I'm working on.

And I'm just kind of picking out from this giant collection of low-hanging fruit and going after them in not a very principled way. That's epic. That sounds so cool. Okay. Sriharsha wrote, are there any scalable solutions like with model-based RL for inferring actions using deep active inference instead of evaluating many policies, which might be computationally intractable? It's related to what we discussed. What would you say?

Yeah. Well, one thing that I'll say is I'm not evaluating many policies. If you think of a neural network as a function approximator, you're like, well, what function is it approximating?

So when someone has a model-based architecture, essentially they're giving you a function. They're saying, I have two functions. I have the model function and I have a planning function. So the model function is going to produce rollouts in some hypothetical environment. And that's a function that's like modeling the environment. And then a planning function is an algorithm that says, given a model function, how do I decide what to do next? And so with model-based RL, you've partitioned the function into two components. And then you try to either learn or engineer them separately. So typically model-based RL, you try to learn the model function, but you keep the inference function algorithmically fixed. In an end-to-end model free RL, you're like, look, I'm not even going to bother separating the two functions. Because if you think about it, keeping them separate, having them separate might give you an easier learning problem. It's easier to learn a decomposed function, to decompose functions than a joint one. Learning a composed function gives you a lot more degrees of freedom. So it's harder to learn, but it also means that sometimes you do more planning, sometimes you do less planning, sometimes as you're planning, you might decide, oh, actually, in this part of the planning space, I want to do a lot more, I want to focus a lot more on rollout. So I'm pretty agnostic to whether the right architecture is model-based versus model-free, and my framework allows you to play with both. And that's a whole research direction of like, okay, what is the right agent architecture? Right now, it's a model-free single neural network that's essentially internally in ways that we don't understand is doing whatever modeling and planning that we want to do. I think it would be really cool to say, hey, will we get better architectures if we break it up into a modeler, like a model and a planner? Can we actually get better performance? But I think these are all empirical questions. I don't think there's an a priori, you know, better architecture for it. I want to play with both. And I think either way, what you're trying to learn is a bunch of heuristics. Awesome. Great points. And super interesting. And so, again, it's the flexibility of active inference as the topology of the compute goes from the sparser, more interpretable, more kind of single node semantic, more definable, etc. on through the dense universal function approximator, open-endedness, many, many, many architectures already existing

and many to come in that area. And then together they're approximating an approximator. And then you have to engineer it pragmatically either way. So then in practice, these considerations like are empirical because they just relate to how you build it. And so it's on your GitHub, like what repo should people look at just to be like clear with that answer? What repos should people look at? What will happen when they just get started with it?

Yeah. Let me show it if you can. Thank you. Yeah.

So can you see this? So the repo is under GitHub slash baby slash meta. And there is there are links to the Discord. Probably the best way to get, I mean, so there is an overview and getting started. I'm in the middle of a big environment rewrite. So some of these things are going to change soon. And if you're actually seriously thinking about working on this, I urge you to reach out to me on Discord or email me because I can help you get started more quickly. There's also a research roadmap here where I just have like a bunch of ideas of things that people can play with that are kind of low hanging fruit. But yeah, the repo is on GitHub, Davy meta.

That's epic. Thank you. Okay. Next questions. Courtney has two questions. First one.

Regarding the question about how you test whether or not has introspection and that you would like help with that research, what skillset are you looking for in collaborators?

I mean, I can use collaborators with a lot of different skill sets. I mean, what I would really love is someone like at like a co-founder collaborative level, someone that thinks about this stuff day and night like I do that would want to basically be like a deep collaborator. So like software engineer, ML experience. So that's like on the most intense like co-founder level. But then there's so much other research to be done.

So I shared this like behavior gallery here. So this, you know, here, you know, these agents are trained under different scenarios.

And then we take them out of context. And we're like, okay, what do they do in an empty room? What do they do in a bigger empty room?

What happens if we, you know, took the take two agents and stick them in a room with finite resources? Like, how do they behave?

Doing this kind of research is requires almost no technical skills. You basically have to think about like, what kind of environment do I want to design? And then you edit some configuration files to be like, okay, let's make, you know, five empty rooms, put two agents in there, put some food, run the simulation, and then measure this thing. So that can be someone with very low technical skills.

Like someone with very low technical capabilities, but just like someone that wants to think critically about like, hey, how do we study this new life form? Or this new intelligence?

I can also use people that are excited to be like technical communicators and like, write and share these ideas and get them into written form to be shared more broadly.

I can use help with people that want to do like find collaborators or do like outreach to other developers around getting these environments used or getting people to help with them.

So I think just like a broad set of skills, but the ones that are the easiest for me to leverage are going to be people with coding and software experience that want to actually help build some of these components.

And that could be just like that. That does not need to be like AI experience, just software being excited to write software is sufficient.

Thank you. Great answer. What over this process has surprised you the most slash what have you learned in this journey so far?

Like about the model output or how it related to some of the other engineering, just any aspect about it.

Yeah, I mean, one thing that and this shouldn't be surprising, but things are hard.

Things take way longer than you think they're going to take.

And it's usually the hard stuff is not some like complex algorithm.

It's just like building stuff, getting it to work.

Before I dove into the world of reinforcement learning, I had no idea how nascent that field is.

Pretty much everyone working on RL is like RL is really hard.

It almost doesn't work.

It's just like it's a very young field.

And so on the one hand, like that was surprising.

It's also really surprising how much of mainstream RL is working on stuff that I have a hard time getting excited about.

Like most of RL is trying to play Pong.

They're trying to, you know, be like, can we play Atari games better than we played than these other algorithms played Atari games?

And that's not fair to the entire field.

But there's very little research going into using reinforcement learning in these complex open-ended environments.

It's like almost no one is working on that.

And RL and then, yeah, I guess those are some surprises for me.

This is just like a green pastures.

Very interesting.

It relates to something I've heard many times in the kind of active area is what are the benchmarks?

So other than just the computational resourcing and all those kinds of performance, like how do you think about benchmarking when arguably that will have open-endedness in certain ways or conditionalities or impossibilities?

Yeah, great question.

And I've given this a lot of thought and I don't have like a perfect answer, but I have a direction.

So I would say two things.

One, I think what I started with is behavior catalog.

But I think one, there's like a behaviorist empirical study that you could do to these agents.

So you could be like, okay, we trained these agents in these open-ended worlds.

Now, if we take them out of their context and actually try to do cognitive tests on them, how well do they do?

You know, can we offer it poison berries in addition to normal berries and will it learn to avoid them?

Can you, in the same way that we try to quantify the intelligence of animals, it's like, okay, can you train it?

Can you talk to it?

Can you convince it?

How much training does it need?

Can it solve, like, does it have memory?

Like if you set it up in a situation where it needs to remember five things, does it work?

What about 10 things?

What about 50?

And so a lot of this is up to the researcher's intelligence rather than the entity's intelligence is like, can we be smart enough to figure out how to test for cognitive ability?

So that's one direction.

And then the other direction is pure competitive fitness.

So if you have two policies, you know, you have two different brains and you have one control 10 agents and the other one control 10 agents, which ones will get more free energy?

So essentially like an ELO style competitive evaluations of policies is the other direction.

And that gets tricky because maybe, you know, in a world where brain A controls one agent and brain B controls 10 agents is going to have a different outcome where brain A controls 10 agents and brain B controls one agent.

So there's still some settings.

You got to test head to head competition under a variety of different settings.

And you can imagine that some do better in resource rich environments where others do better in resource poor environments.

So you have to be careful around trying to quantify superiority.

But I think overall we should get to a point where you can say, Hey, look under a lot of conditions, this new brain that we trained is able to outperform this old brain that we trained in a competitive setting.

And then we have a bunch of evals that are measuring things like memory, trainability, ability to solve problems and not get stuck at local optima.

And it's performing better on this set of evals.

So I think it is a quantifiable approach.

It's just, it's not, it's not trivial.

Yeah.

Great answer.

How do they do it in other systems?

Okay.

Here's a question from board guy.

Do you make, there's a few pieces of clarification.

Do you make a distinction between concepts of piecewise versus nonlinear learning in the context of emergence?

More specifically, do you think of these types of measures as a lack of knowledge about what the system is doing on our end?

As in the nonlinearity is simply the piecewise measure not tested, right?

Do you think of these two measures, the continuous nonlinear and the piecewise?

Do you think of these two measures as representative as two surely different learning rules?

Or are we simply testing the model in the wrong ways?

I don't know if I followed all of that, but feel free to rephrase the question if I'm not answering what you're asking.

But the part that really resonated for me is, and I've moved really heavy in this direction.

A lot of what we're talking about is not, I don't think there is, you know, what is the system doing?

What is its learning rule?

There is, what are our beliefs about its learning rule?

So a lot of what we're talking about is as observers, what do we, how do we model the system?

And we could model it as having a learning rule A with like a lot of competence or learning rule B with less competence or a set of learning rules that are interacting in nonlinear ways.

So partially this is the way we, we would talk about the model rather than what is the model doing itself.

And so I think with that formulation, I'm, I'm not that, I guess, interested in like, what is, like, what is the model actually doing?

I'm more interested in like, what can we measure about this model's behavior?

So what, when we look at the behavior of an agent, is it acting in a way that we think is intelligent?

And maybe that's what you're asking rather than, you know, what is the neural network doing internally and how much?

And it's interesting because the dynamics are highly nonlinear.

What the network is doing is reacting to what the other agents are doing, which is also what the network is doing.

So it's essentially conditioned on its own conditioning.

So I don't think there is any kind of like tractable linear way to decouple these behaviors.

It's highly recurrent and highly nonlinear.

That's a great answer.

A few other aspects is that the concept of figure and ground, kind of like agent-based modeling in an environment that is like a grid world environment.

That is one topology.

And so there's kind of that statistical dense click that is the input output, whether that's trained end to end or in the two separate sections.

And then there's that's like a bumper car with either the niche read and write and or the long scale communications like you brought in later.

So then then especially when you bring in all these fluctuating knobs, like let's just say that we learn certain behavioral policies when we're in a bright room versus when we can't see.

Then environmental discontinuity, which is something that the experimenter has done.

So again, reflect that relational piece.

It would discontinuously change it.

But then it's just like, yes, but the neural networks, continuous value numbers, et cetera.

It's like that's like a deflationary account because we already know what structure our map approximator is.

It's just the statistics.

So it's like interesting about is this like applying active inference with reinforcement learning or one or the other.

And then it's like, but you just look at the open source code and there's like and then it could be composed another way.

And having the ability to swap out different kinds of methods for each other helps highlight like that flexibility.

He wrote, I guess the way you think about these has implications about whether you can simply scale it up if you add more variables.

I guess more variables.

There's two places to add variables.

One place is in the complexity of the environment.

So as we add more behavior, like more objects and more diversity into the environment, there is essentially the environment becomes more complex.

But really the main source of variables is the number of parameters in the models.

And I assume that's what you're mostly talking about.

And really, again, from the agent's perspective, the environment is all of the other models on the other side of you.

So for every agent, you have a neural network with, say, 10 million parameters.

But your environment is 50 other agents, each of which has 10 million parameters.

And what you do and what you perceive is going to depend on what those networks are doing.

And so, yeah, the idea is to scale this by training larger and larger neural networks.

And the larger the neural network, the more parameters your neural network has, the more it's capable, the more complexity it's capable of representing.

But the harder it is for you to train it and the more compute you need.

So, yeah, the whole point of this project is so in RL, typically the models we train are a few hundred thousand to maybe a few million parameters.

And language models were training, you know, 70 billion, a hundred billion parameter networks.

And the idea is to try to make infrastructure and an environment that is so fast for generating training data that we can train of these really large high parameter neural networks so that they can generalize over dealing with really complex environmental spaces.

And essentially so that they can learn complex learning algorithms themselves themselves.

So, yeah, the whole point is to add more and more variables to the neural network.

Great point.

And that highlights that in the agent versus the niche situation is averted because the resourcing of the niche is so much greater.

Because it's only 10 versus versus if there's a hundred other agents or something, then it's only 1% of the resources per agent.

So that even that kind of structurally prepares a very different environment than a dyadic, even dyadic collaborative stationary training paradigm.

So there's a lot there with the movement and also the very cool.

So any last comments?

Otherwise, I hope this has been exciting for people to learn more and to look at the GitHub and get involved.

Anything else?

Oh, sorry.

Are you asking me or are you asking me?

Do you have anything else?

Oh, no.

Thank you so much for your time.

Feel free to join the Discord.

Here, I'll bring the link back up.

And also feel free to message me if you want to chat or collaborate in any way.

Yeah.

I mean, for me, this has been a really fun project and I'm just like excited to keep working on it and finding other people that want to work on it with me.

Epic.

Okay.

Thank you, David.

Bye.

Yeah.

Thank you, Daniel.

Bye.

Bye.

Thank you.